CityTech TTP Summer Bootcamp

Express and Pair Programming

2026-06-16

Agenda

1. Review
2. Follow Up: React keys
3. Complex `fetch` and HTTP Headers
4. API Practice: JSONPlaceholder and SpaceTraders
5. Building an API with Express
6. Middleware
7. Pair Programming and AI

Review

What are the most important things you remember from yesterday?

Follow Up: React `key`

When you render a list with `.map()`, React wants a **`key`** on each item:

```
{items.map((item) => <li key={item.id}>{item.name}</li>)}
```

- **Why:** `key` lets React tell items apart between renders — so it updates only what changed and keeps each item's state correct
- **Use a stable, unique id** (e.g. `item.id`) — it stays with the item even if the list reorders
- **Avoid the array index** — if items are added, removed, or reordered, indexes shift and React mismatches them (stale state, subtle bugs)

Follow Up: Terminology — Endpoint

An **endpoint** is a specific **address (URL) your API exposes** — a place a client can send a request to.

- e.g. `https://api.example.com/books/42` is an endpoint
- Pair it with a **method** to describe a full operation: `GET /books/42`
- In Express, every **route** you define is an endpoint
- "Hit the `/books` endpoint" just means *send a request to that path*
- *Endpoint*, *route*, and *path* get used loosely / interchangeably in practice

Complex Node.js `fetch` Calls

A `GET` just reads. A write like `POST` adds a `method`, JSON `headers`, and a stringified `body` (the payload):

```
// GET — read a post
const res = await fetch("https://jsonplaceholder.typicode.com/posts/1");
const post = await res.json();

// POST — create a post
await fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Hi", userId: 1 }),
});
```

HTTP Headers

Headers are key–value metadata sent with a request (and its response) — info *about* the request, separate from the body.

```
Content-Type: application/json
Authorization: Bearer <token>
```

- `Content-Type` — the format of your body (e.g. `application/json`)
- `Accept` — the format you want back
- `Authorization` — your credentials, e.g. `Bearer <token>` — **auth**
- `X-API-Key` — an API key, another **auth** style
- `Cookie` — session data sent back to the server — also **auth**

Code Along: JSONPlaceholder

Write a Node script that talks to **JSONPlaceholder** — https://jsonplaceholder.typicode.com

- **GET** `/posts/1` and print its title
- **POST** to `/posts` and log the response
- Try **PATCH** and **DELETE** on `/posts/1`

It accepts writes and returns realistic responses — but doesn't actually persist them.

More API Practice

- Play the **SpaceTraders** game from yesterday — https://spacetraders.io/
- **Tiered learning:** write a **Node.js script** to automate parts of the game — or the whole thing!
- We'll create a class **leaderboard** of players

Express: a Node Server

Express is a minimal web framework for **Node.js** — it lets you build a server that responds to HTTP requests (your own API).

A **route** says: *when a request matches this method + path, run this handler.*

- **Method + path** — e.g. `GET /add/:a/:b`, `GET /multiply/:a/:b`, `GET /square/:n` (`:a`, `:b`, `:n` are **URL parameters**)
- **Handler** — a function `(req, res) => { … }`:
  - `req` — the incoming request (params, query, body, headers)
  - `res` — how you reply (a status code + data, usually `res.json(...)`)
- Express checks routes **top to bottom** and runs the **first match**

Express: Example

```
import express from "express";
const app = express();

app.get("/", (req, res) => {
  res.send("Hello, world!");
});

app.listen(3000);
```

Visit `http://localhost:3000/` → **"Hello, world!"** — a quick smoke test that your server runs.

Express: Setting Up and Running

```
yarn init -y              # set up package.json — only once per project
yarn add express          # install Express into your project
node server.js            # start your server
```

- Put your code in a file (e.g. `server.js`), then run it with **`node`**
- It keeps running — leave it up; visit `http://localhost:3000` to hit it
- **`import express`** needs `"type": "module"` in `package.json` (or a `.mjs` file)
- **Ctrl-C** stops it; restart to pick up code changes — or use **`nodemon`** to auto-restart

Express Is Just a Running Program

An Express server is a **long-running Node program** — you start it and it keeps running, waiting for requests.

- It's **no different from a Node.js script** — just one that doesn't exit
- Anything it stores lives in **RAM** (plain variables)
- So when the server **restarts, that data is gone** — we can add a database later

Express: a POST Route

```
app.use(express.json()); // parse JSON request bodies

app.post("/books", (req, res) => {
  const newBook = req.body;         // the JSON payload
  res.status(201).json(newBook);
});
```

- `req.body` holds the posted JSON — needs `express.json()`
- Reply **`201 Created`** with the new resource

Express: One URL Parameter

```
app.get("/books/:id", (req, res) => {
  const { id } = req.params;
  res.json({ id, title: "Dune" });
});
```

- `:id` in the path → `req.params.id`
- `GET /books/7` → `req.params.id` is `"7"` (a string)

Express: a PATCH Route

```
app.patch("/books/:id", (req, res) => {
  const { id } = req.params;
  const updates = req.body;          // fields to change
  // ...apply the updates to book `id`...
  res.json({ id, ...updates });
});
```

- **PATCH** updates *part* of a resource (**PUT** replaces the whole thing)
- Combines a URL param (`:id`) with a JSON body (`req.body`)

Express: a DELETE Route

```
app.delete("/books/:id", (req, res) => {
  // ...remove the book with this id...
  res.status(204).end();           // No Content
});
```

- The `:id` comes in on `req.params`
- **`204 No Content`** is common after a successful delete

Express: Two URL Parameters

```
app.get("/books/:bookId/reviews/:reviewId", (req, res) => {
  const { bookId, reviewId } = req.params;
  res.json({ bookId, reviewId });
});
```

- Each `:name` segment becomes a key on `req.params`
- `GET /books/7/reviews/3` → `{ bookId: "7", reviewId: "3" }`

Express: Query Parameters

```
// GET /books?sort=title&limit=10
app.get("/books", (req, res) => {
  const { sort, limit } = req.query;
  res.json({ sort, limit });
});
```

- Everything after `?` lands on **`req.query`** as key–value pairs
- **Path params** (`:id`) say *which* resource; **query params** (`?sort=`) tweak *how* you read a list — filter, sort, paginate
- Like path params, the values arrive as **strings**

Express: Returning an Error

```
app.get("/books/:id", (req, res) => {
  const book = findBook(req.params.id);
  if (!book) {
    return res.status(404).json({ error: "Book not found" });
  }
  res.json(book);
});
```

- Choose a fitting **status code** (`404` , `400` , …) and return a message
- `return` so the handler stops
- For unexpected errors you can also `throw` — Express routes it to an error handler

Aside: Method Chaining

`res.status(404).json(...)` calls one method right on the result of the previous — that's **method chaining**.

```
res.status(404).json({ error: "Not found" });
```

- It works because `res.status(404)` **returns** `res`, so you can keep calling on it
- Reads left-to-right as a pipeline — same as writing each call on its own line

Express: Middleware

Middleware is a function Express runs **in between** receiving a request and sending the response.

```
app.use(express.json());   // runs on every request
```

- Signature `(req, res, next)` — it can read or change `req` / `res`, then calls `next()` to pass control along
- Express runs them in a **pipeline**, top to bottom, **before your route handler**

```
request → [express.json()] → [logger] → [auth] → your route → response
```

- Common ones: `express.json()`, `cors`, a request logger, an auth check

Middleware vs. Importing a React Library

Both pull in **reusable code someone else wrote** — often installed the same way (`npm install`, then `import`). The real difference is **who calls it, and when.**

Importing a React library — *you* call it, explicitly, right where you want it:

```
import { format } from "date-fns";
format(new Date(), "yyyy-MM-dd");   // you decide when this runs
```

Middleware — you **register** it once, and *Express* calls it for you, automatically, on every request. If registering multiple, order matters!

```
app.use(express.json());   // you never call it yourself
```

Middleware vs. Importing a React Library

- **React import → you're in control:** you invoke the function at a specific spot in your code
- **Middleware → inversion of control:** you hand the function to the framework, and *it* runs it for you — automatically, in a pipeline, before your handler
- A React component renders **where you place it**; middleware runs on **every request** whether or not you "use" it anywhere
- **Order matters for middleware, not for imports** — where you put an `import` is cosmetic, but middleware order changes behavior
- Same idea both ways: don't reinvent it — but middleware plugs into Express's **request lifecycle**, not your own call stack

Code Along: Express Calculator

Build a calculator API with these routes:

- Two params: `add`, `subtract`, `multiply`, `divide`
- One param: `square`, `cube`

Handle the edge cases:

- a parameter that **isn't a number** → respond with an error (e.g. `400`)
- **divide by 0** → respond with an error

Pair Programming

Two people, one task, working together in real time.

- **Driver** — types the code
- **Navigator** — reviews, thinks ahead, spots issues, suggests direction
- **Switch roles often**
- Benefits: fewer bugs, shared knowledge, faster unblocking

Fishbowl Demo: Deck of Cards API

Reviewing solutions from yesterday's challenge — in a **fishbowl**.

AI as Your Pair Programmer

Treat the AI like a **partner you collaborate with** — not an autopilot.

- **You stay the lead:** set the goal, review every suggestion, decide what to keep
- Use it to brainstorm, explain, draft, and debug — **trust but verify**
- Use it **ethically** — disclose your use, and don't pass off code you don't understand as your own
- vs. *just letting AI code*: you'd ship code you don't understand, can't debug, and can't vouch for
- Rule of thumb: **never commit code you can't explain**

Using CUNY Copilot alongside VS Code

CUNY's licensed AI is **Microsoft Copilot** (data-protected) — it runs in the **browser**, not inside VS Code. So use them **side by side**:

- Open **Copilot Chat** in a browser, signed in with your **CUNY account**
- Keep **VS Code** open beside it
- Copy code / errors into Copilot, paste suggestions back — then **review and verify**
- Because it's CUNY-licensed, your prompts and files **aren't used to train** the model

> Heads-up: the in-editor "agent" is GitHub Copilot — a separate product CUNY doesn't license.

Demo: Calculator Express Server

A quick **demo** of building the calculator **server** with Express.

- Implement the `add`, `subtract`, `multiply`, and `divide` endpoints

Demo: Calculator Client via AI

A quick **demo** of building a calculator **client** with AI as a pair programmer.

- A small **React app** that takes two numbers and an operation
- Calls the Express endpoint with `fetch` (e.g. `/add/3/4`) and shows the result
- Also have the AI generate **standalone Swagger / OpenAPI docs** for the calculator API
- Watch how to **lead** the AI: describe the goal, review the code, **trust but verify**
- ...and disclose the AI use

Coding Practice: Calculator Express App

Re-implement the `add`, `subtract`, `multiply`, and `divide` routes we built — **by hand**. Try not to copy-paste or use AI.

Tiered learning:

- Add **`is-even`**, **`is-odd`**, **`absolute-value`**, and **`exponent`** routes — use **URL parameters**
- Add **`sum`**, **`average`**, **`min`**, and **`max`** routes — use **query parameters**